

RisingWave vs Ververica: Platform Decision Brief

A platform-selection brief for streaming workloads similar to those exercised in this repository. The repository is referenced as a worked example, not as the subject of the decision.

The choice considered here is between **RisingWave** (streaming database, managed compaction, queryable materialized views) and **Ververica** (commercial managed Apache Flink platform from Ververica / Alibaba: Flink as the engine, with deployment, lifecycle, savepoints, autoscaling, and an SQL-first UI handled by the platform).

This is **not** a comparison between RisingWave and self-hosted open-source Flink. Many of the operational concerns that apply to bare Flink are mitigated by Ververica. The brief focuses on what remains.

1. Executive Summary

Recommendation: Choose RisingWave as the default for the workload patterns in Section 2. Choose Ververica when its specific strengths apply (Section 5 / Section 6).

For the workloads covered here, the architectural gap that matters is not operational tooling — Ververica handles deployment, savepoints, and lifecycle well — but the **interaction model**. RisingWave gives you a queryable streaming database; Ververica gives you a managed Flink job that emits sinks. When consumers (ML, analytics, ops) need to query live derived state, RisingWave eliminates an entire serving layer.

The two real reasons to pick Ververica are: (a) workloads beyond declarative SQL — large keyed state, CEP, custom operators — which Flink handles better; or (b) an organizational commitment to Apache Flink as a strategic platform with portability beyond a single vendor.

This brief is intentionally opinionated. Section 5 lists where Ververica is genuinely the right answer, and Section 7 names the honest risks of each choice.

2. Workload Patterns This Brief Covers

The recommendation applies to workloads dominated by these patterns:

- **Windowed aggregation over Kafka** with seconds-to-low-minutes freshness (e.g., funnels, conversion rates, rolling counters).
- **Stream enrichment and joins** between event streams and reference data, with results either pushed to a sink or queried directly.
- **Sink fan-out** to Kafka, an Iceberg lake, and an OLTP database (e.g., PostgreSQL) from the same derived dataset.
- **Iceberg as a source for slowly changing reference data** — a view reads a slowly changing dimension or reference dataset from Iceberg and joins it against streaming events.

- **Database ingestion of reference data** — reading tables from an operational database (e.g., PostgreSQL) on a schedule or as a bounded snapshot to enrich streams. Not CDC — the source is queried, not subscribed to.
- **Live feature serving** where ML or operational consumers need to read fresh derived state on demand, not just consume a sink.
- **Updates and corrections to derived data** where changes propagate through derived artifacts and into the lake.
- **Source-side updates, deletes, and corrections (GDPR-style) with automatic recalculation** — the engine holds source data in mutable tables; an `UPDATE` or `DELETE` on a source row (e.g., a refund, a privacy-driven erasure, a back-dated fix) propagates automatically through derived materialized views and into the lake via upsert sinks. No replay job, no manual recompute.

The recommendation **does not** automatically apply to:

- Very large keyed state (multi-terabyte per job) with strict per-key isolation.
- CEP / pattern detection across long windows.
- Workloads requiring custom operators, embedded ML inference, or non-SQL stateful logic.
- Multi-stage DAGs with bespoke failure-recovery semantics.

These cases are addressed in Section 5.

3. Recommendation in One Table

Decision dimension	Pick RisingWave	Pick Ververica
Primary interaction model	Consumers issue SQL queries against live derived state	Consumers consume sinks (Kafka / Iceberg / DB) and never query the engine
Team skill profile	SQL-first analytics or data engineering team	SQL-first is fine on Ververica too, but <code>DataStream</code> / Java is available when needed
State size per job	Up to roughly hundreds of GB per job, mostly aggregations and joins	Multi-TB keyed state, custom state backends, fine-grained tuning
Job complexity	Declarative SQL pipelines with windows and joins	Custom operators, CEP, multi-stage DAGs, embedded inference
Iceberg lake operations	Compaction, expiration, and rewrites should be handled by the platform	Team is willing to operate Iceberg maintenance separately (Spark, Trino, custom)
Time-to-first-production	Weeks	Weeks-to-months — Ververica is far closer to RisingWave on this than bare Flink
Strategic intent	Stream processing is a means to an end	Standardize on Apache Flink as a long-term platform, with managed UX from Ververica

If most rows in a candidate workload point left, RisingWave is the default. If a meaningful number point right, Section 6 lays out when Ververica becomes the better choice.

4. Capability Parity

For most of the workloads in scope, both platforms can produce the same business result. The differences narrow significantly with Ververica compared to bare Flink — operations and deployment are no longer where the gap lives.

Capability	RisingWave	Ververica	Notes
Windowed aggregations from Kafka	Native, declarative SQL	Native, Flink SQL or DataStream	Parity.
Joins between streams and reference data	Native, SQL	Native, SQL or DataStream	Parity.
Sink to Kafka	Native	Native	Parity.
Sink to PostgreSQL / OLTP	Native upsert	Native via JDBC connector	Parity.
Sink to Iceberg	Native, with managed compaction	Native via Iceberg connector; maintenance still your responsibility	Ververica runs the Flink job for you, but does not run Iceberg maintenance (compaction, expiration, rewrites). RisingWave bundles a compactor service.
Iceberg as a source (slowly changing reference data)	Native source; the table can be joined against streams as a regular SQL relation, with periodic refresh of the snapshot	Native via Iceberg connector; can be modeled as a lookup join or a bounded source refreshed on schedule	Both viable. RisingWave's value is that the join is just SQL inside the same engine that owns streaming state. Ververica requires explicit modeling of how often to re-read snapshots and how to merge with active state.
Database ingestion (snapshot / batch read of reference tables)	Native source connectors with scheduled or bounded reads; result is a regular SQL relation	Native via JDBC connector or scheduled bounded source	Both viable; RisingWave keeps the join in one engine without an external refresh pipeline.
Updates / deletes through derived state	First-class via tables and materialized views	Requires explicit changelog modeling and care around retract semantics in Flink SQL	RisingWave is meaningfully simpler.

Source-side UPDATE / DELETE with automatic recalculation (GDPR, refunds, back-dated fixes)	First-class. Source is a mutable table; an UPDATE or DELETE propagates through every dependent materialized view and out to upsert sinks (Iceberg, Postgres) automatically. No replay job.	Not native. Source corrections must be emitted as changelog events into Kafka (or fed via a dedicated CDC source), and every downstream operator and sink must be modeled to handle retracts. Custom backfill jobs are typically required for already-emitted aggregates.	One of the largest operational gaps. RisingWave handles this as a database operation; Ververica handles it as a stream-processing engineering project.
Live SQL query on derived state	First-class — derived data is queryable as a database	Not native — requires sinking to a serving store (Postgres, Iceberg, KV)	This is the primary functional gap and the strongest reason to prefer RisingWave for in-scope workloads.
Custom operators / non-SQL logic	Limited (SQL + UDFs)	Strong (DataStream API, ProcessFunction)	Ververica is meaningfully stronger.
Complex Event Processing	Not a focus	Mature library (FlinkCEP)	Ververica is the right tool here.
Exactly-once across heterogeneous sinks	Per-connector guarantees	Two-phase commit framework, broader coverage	Ververica is more rigorous when this is critical.
Replay and savepoints	Limited operator surface; relies on source replay	First-class savepoints and state migration, surfaced in the Ververica UI	Ververica makes this easy without bespoke tooling.
Job lifecycle, deployment, autoscaling	Handled by RisingWave (cluster)	Handled by Ververica (per-job, with a UI)	Parity in <i>who operates it</i> : Ververica closes the gap that bare Flink leaves open.

5. Where Ververica Is Genuinely the Right Choice

To keep this brief credible, it is worth stating clearly when Ververica should win.

- **Multi-terabyte keyed state per job**, where RocksDB tuning, incremental checkpoints, and state TTL strategies are necessary.
- **Complex Event Processing** — pattern detection, sequence matching, fraud-style rules across long windows.
- **Custom operators or embedded ML inference** in the streaming path, where SQL is a poor fit.
- **Multi-stage DAGs** with bespoke failure semantics, replay rules, or per-stage backpressure handling.
- **Strict exactly-once across heterogeneous sinks** using two-phase commit, especially for financial or compliance-critical pipelines.
- **Standardization on Apache Flink** as a strategic platform, valuing portability across vendors (Confluent, AWS MSF, self-hosted) over a single-vendor managed product.
- **Existing Ververica or Flink footprint** for other workloads — the marginal cost of one more job

is low and the platform team already exists.

If a workload genuinely fits one of these, Ververica is the right call regardless of the rest of this brief.

6. When to Pick Ververica Even For In-Scope Workloads

Even within the workload patterns in Section 2, choose Ververica if **any** of the following holds:

1. The architectural gap in Section 4 (live SQL query on derived state) is not a hard requirement — every consumer is happy reading from a sink — *and* there is strategic value in standardizing on Flink.
2. There is a credible pipeline of stateful jobs where some will eventually need DataStream-level control, and you prefer one platform across all of them.
3. The organization already operates Ververica or Flink and the cost of adding a streaming database alongside is not justified for this workload alone.
4. Lake-side maintenance (Iceberg compaction, expiration, rewrites) is already owned by another team or platform.

If condition 1 is missing, the live-SQL gap usually dominates the decision for the workloads in Section 2.

7. Risks of Each Choice

Both choices carry risk. Naming them honestly is part of a defensible decision.

Risks of choosing RisingWave

Risk	Mitigation
Single-vendor product , smaller ecosystem and contributor base than the Flink ecosystem behind Ververica.	Treat the project as a strategic dependency: track release cadence, document an exit path (Section 9), use the commercial offering if production-critical.
Smaller hiring market for RisingWave-specific expertise.	The skill is mostly SQL plus operational basics, so cross-training is feasible; deep RisingWave internals expertise is rare.
Scale ceiling is less battle-tested than Flink for very large state or very high throughput.	Validate the upper bound for your workload during the POC; have a documented threshold above which you would reconsider.
Connector and ecosystem maturity is narrower than Flink's.	Confirm every required source and sink is supported with the semantics you need before committing.
Custom logic ceiling : workloads that grow beyond SQL + UDFs are awkward.	Identify candidate workloads early and either keep them on a different platform or accept future migration.

Risks of choosing Ververica

Risk	Mitigation
------	------------

No live SQL query surface: every consumer that wants fresh derived state needs a serving store (Postgres / Iceberg / KV).	Decide the serving store up front; do not let each consumer pick its own. Quantify the operational cost of that extra layer before committing.
Iceberg maintenance is unbundled: compaction, expiration, and rewrites must be implemented and operated separately from Ververica.	Plan a maintenance strategy (Spark jobs, Trino, scheduled rewrites) before going to production.
Commercial licensing cost: Ververica is a paid platform, typically priced per managed instance / vCPU and meaningful at small scale.	Get an actual quote for your expected workload before deciding; factor ongoing license cost into the cost model.
Vendor lock-in to Ververica's tooling, even though the underlying engine is OSS Flink.	Keep jobs in Flink SQL or portable DataStream code; avoid Ververica-specific extensions where possible.
Higher onboarding cost for SQL-first analytics teams compared to RisingWave.	Use Flink SQL via the Ververica UI; reserve DataStream for jobs that genuinely need it.
Job-centric, not database-centric: every dataset is a job artifact, not a queryable relation.	Ensure this matches the team's mental model and the consumer architecture before adopting.

8. Cost and Operating Model

Cost is workload-dependent, but the *shape* of the cost differs in predictable ways.

Cost dimension	RisingWave	Ververica
Compute infrastructure	Comparable per-job for in-scope workloads	Comparable per-job; can be lower at very high scale with tuning
Platform engineering	Low — most platform concerns are internal to RisingWave	Low — Ververica handles deployment, savepoints, autoscaling, lifecycle
Software licensing	OSS, or RisingWave Cloud subscription	Commercial license (Ververica Platform / Ververica Cloud) — paid product
Iceberg maintenance	Bundled (compactor service)	Unbundled (separate Spark / Trino / custom jobs)
Serving layer for live queries	Not needed — engine is queryable	Required — additional Postgres / Iceberg / KV component to operate
Connector operations	Built into the engine	Managed by Ververica with operator surface
On-call surface	Engine + sources + sinks	Ververica jobs + sources + sinks + serving layer + Iceberg maintenance

The headline: with Ververica, the **operational** cost gap to RisingWave largely closes (Ververica handles what bare Flink does not). The remaining gaps are **commercial licensing**, **the extra serving layer needed to expose live state**, and **Iceberg maintenance**. Conversely, RisingWave concentrates risk in a single vendor with a smaller ecosystem.

9. Exit Paths

A platform choice should be defensible against the question "what if we outgrow it?"

If you choose RisingWave and outgrow it:

- Most pipelines are SQL — translatable to Flink SQL with effort but without a full rewrite.
- Sinks (Kafka, Iceberg, Postgres) are standard formats — consumers do not have to change.
- The hardest migration is anything that depends on RisingWave-specific UDFs or its query-as-database serving model.

If you choose Ververica and want to migrate off:

- Flink SQL jobs are portable to other Flink runtimes (Confluent Cloud for Flink, AWS MSF, self-hosted) — Ververica is not a one-way door at the engine level.
- DataStream API jobs are portable to any Flink runtime but not to other engines.
- State (savepoints) is portable across Flink runtimes; you typically cannot migrate state to a non-Flink engine.
- Ververica-specific platform tooling (deployments, dashboards, autoscaling config) is non-portable.

Net: Ververica's underlying engine portability is a real advantage. RisingWave's portability story exists only at the SQL / sink-format level.

10. Decision Rule

Use RisingWave by default for the workload patterns in Section 2.

Choose Ververica instead when *any* of the following holds:

- The workload matches one of the cases in Section 5 (large state, CEP, custom operators, etc.).
- Live SQL query on derived state is genuinely not needed, *and* there is strategic value in standardizing on Flink (Section 6, condition 1).
- The organization already operates Ververica or Flink and the marginal cost of one more job is low.
- Long-term portability across Flink runtimes is a hard requirement (regulatory, vendor diversification).

In all other cases, RisingWave is the recommended choice.

Appendix A: Worked Example — This Repository

The repository this brief lives in is a useful concrete example. It exercises the workload patterns from Section 2:

- **Streaming compute:** views and materialized views over Kafka topics (`page_views`, `cart_events`, `purchases`) producing derived datasets such as funnel metrics, training features, and per-user summaries.
- **Sink fan-out:** the same derived data is published to Kafka (for a dashboard), Iceberg (for analytics and offline ML), and PostgreSQL (for serving).

- **Queryable serving:** ML training and online learning code paths query RisingWave directly for fresh feature data instead of going through a separate serving store.
- **Updates to derived data:** a "Hermes" feature flow (named after the messenger pattern) demonstrates updates and deletes propagating through derived artifacts and into Iceberg.
- **Source-side corrections (GDPR-style) with automatic recompute:** the Hermes flow keeps source data in RisingWave tables (`tbl_hermes_page`, `tbl_hermes_cart`, `tbl_hermes_purchase`) that accept `UPDATE` and `DELETE`. A refund, a privacy-driven erasure, or a back-dated correction issued directly as SQL flows automatically through the `hermes_features` materialized view and out to Iceberg via an upsert sink. There is no separate backfill or replay pipeline.
- **Iceberg as a slowly-changing source:** a RisingWave view reads reference data from an Iceberg table and joins it against streaming events, with the snapshot refreshed periodically. The join is plain SQL in the same engine that owns the streaming state, so there is no separate refresh pipeline to operate.

For this example, the recommendation matches the general one: build on RisingWave. The decisive factor is the queryable-serving pattern: ML and analytics consumers reading live derived state directly. Replicating that on Ververica would require introducing and operating a serving layer (Postgres or KV) on top of the Flink job, plus an Iceberg maintenance strategy. The Hermes source-correction pattern compounds this: on Ververica it would require remodeling sources as changelog streams, designing retract-safe operators, and likely building a separate backfill job for previously emitted aggregates. There is no current evidence of state size or pattern complexity that would push the workload into the Ververica-favoring cases in Section 5.

Appendix B: Proof-of-Concept Plan

A narrowly scoped Ververica POC is documented separately in [docs/VERVERICA_POC_PLAN.md](#). It is intended for organizations that want side-by-side evidence before committing.